

Compiladores 2026–2

Práctica 4

Analizadores sintácticos con Yacc/Bison

Prof. Ariel Adara Mercado Martínez

Ayud. Carlos Gerardo Acosta Hernández

Adud. Luis Mario Escobar Rosales

Rodrigo André Decuir Fuentes

No. Cuenta: 320022711

ojo@ciencias.unam.mx

Fecha de entrega: 12 de mayo



Ejercicios

Los ejercicios 1–4 piden extender la calculadora del ejemplo. El ejercicio 5 pide construir el analizador léxico-sintáctico completo para la gramática G .

1. Modificar el código fuente para que la calculadora pueda operar con flotantes, no solamente enteros.

Respuesta: Se modificó el `%union` en `calculadora.yy` para usar `double` en lugar de `int` y se amplió la expresión regular del lexer para reconocer la forma `DIG+.DIG+`. La acción del lexer llama `atof(yytext)` y marca `tipo = 2` (flotante). Las acciones semánticas de `expresion` ahora propagan `double` mediante operaciones de punto flotante, de modo que `1.5 + 2.5` produce 4.0 en lugar de 4 (entero).

2. Modificar el código fuente para que la calculadora pueda reconocer más líneas con expresiones en el archivo de entrada.

Respuesta: Se agregó la producción

```
program : program line
        | line
        ;
```

como símbolo inicial (`%start program`), donde cada `line` es una `expresion` seguida de `'n'` o EOF. El lexer devuelve el token `NEWLINE` al encontrar `'n'` y el analizador lo usa como delimitador de cada expresión, permitiendo archivos con múltiples líneas.

3. Modificar el código fuente para que la calculadora pueda operar con restas y divisiones.

Respuesta: Se declararon los tokens `MENOS` y `DIV` en el parser con la misma precedencia y asociatividad que `MAS` y `MUL` (`%left MENOS` al mismo nivel que `MAS`, `%left DIV` al mismo nivel que `MUL`). Las reglas gramaticales se extendieron:

```
expresion : expresion MAS expresion { $$.$dval = $1.$dval + $3.$dval; }
          | expresion MENOS expresion { $$.$dval = $1.$dval - $3.$dval; }
          | expresion MUL expresion { $$.$dval = $1.$dval * $3.$dval; }
          | expresion DIV expresion {
              if ($3.$dval == 0) { yyerror("division por cero"); }
              else { $$.$dval = $1.$dval / $3.$dval; }
          }
          | PARIZQ expresion PARDER { $$ = $2; }
          | NUM { $$ = $1; }
          ;
```

El lexer reconoce `"-"` y `"/"` y devuelve los tokens correspondientes.

4. Modificar el código fuente para que la calculadora admita números negativos.

Respuesta: Se añadió la producción unaria

```
factor : MENOS factor %prec UMINUS { $$.$dval = -$2.$dval; }
```

con una precedencia especial UMINUS definida con %right UMINUS a un nivel superior a MUL y DIV, para que el operador unario - tenga mayor prioridad que los operadores binarios. Así, $-3 * 2$ se evalúa como $(-3) * 2 = -6$.

5. Para la gramática $G = (N, \Sigma, P, S)$ descrita por las siguientes producciones:

```
P = {
    programa -> declaraciones sentencias
    declaraciones -> declaraciones declaracion | declaracion
    declaracion -> tipo lista-var ;
    tipo -> int | float
    lista_var -> lista_var , identificador | identificador
    sentencias -> sentencias sentencia | sentencia
    sentencia -> identificador = expresion ;
                | if ( expresion ) { sentencias } else { sentencias }
                | while ( expresion ) { sentencias }
    expresion -> expresion + expresion | expresion - expresion
                | expresion * expresion | expresion / expresion
                | identificador | numero | ( expresion )
}
```

Crear una carpeta llamada C_1 para alojar el analizador léxico y el analizador sintáctico de dicha gramática.

- a. Basándose en la estructura del código de la calculadora, transcribir el archivo de Flex de la práctica 2 en un archivo de Flex.

Respuesta: Listo. El archivo se encuentra en C_1/src/lexer.ll. Reconoce las mismas palabras reservadas (int, float, if, else, while), identificadores de hasta 32 caracteres, enteros, flotantes (DIG+.DIG+) y todos los operadores y delimitadores de la gramática. Cada acción léxica imprime en pantalla el terminal detectado.

- b. Escribir en un archivo de Bison la gramática definida.

Respuesta: Listo. El archivo se encuentra en C_1/src/parser.yy. La gramática se transformó previamente para eliminar ambigüedad y recursividad izquierda, obteniendo

G' : programa $\rightarrow$ declaraciones sentencias
 declaraciones $\rightarrow$ declaracion declaraciones'
 declaraciones' $\rightarrow$ declaracion declaraciones' | ϵ
 declaracion $\rightarrow$ tipo lista_var ;
 tipo $\rightarrow$ int | float
 lista_var $\rightarrow$ identificador lista_var'
 lista_var' $\rightarrow$, identificador lista_var' | ϵ
 sentencias $\rightarrow$ sentencia sentencias'
 sentencias' $\rightarrow$ sentencia sentencias' | ϵ
 sentencia $\rightarrow$ identificador = expresion ;
 | if (expresion) sentencias else sentencias
 | while (expresion) sentencias
 expresion $\rightarrow$ termino expresion'
 expresion' $\rightarrow$ + termino expresion' o - termino expresion' | ϵ

```

termino -> factor termino'
termino' -> * factor termino' | / factor termino' | ε
factor -> identificador | numero | - factor | ( expresion )

```

Se declaró el %union con el campo `numero` (que contiene `dval` y `tipo`) y `sval` para identificadores. Los tokens se anotaron con los campos correspondientes y los no-terminales con el tipo `attr`.

- c. Proponer un mensaje de error sintáctico más explícito.

Respuesta: Se sobrescribió la función `C1::Parser::error()` para que imprima:

```
Error sintáctico: <mensaje de Bison>
```

Bison incluye en el mensaje el token inesperado y su posición, lo cual es considerablemente más informativo que el simple "syntax error" predeterminado.

- d. Proponer archivos de prueba con derivaciones válidas.

Respuesta: El archivo `C_1/src/prueba` contiene una derivación válida con declaraciones de variables enteras y flotantes, asignaciones con expresiones aritméticas (incluyendo negativos), una sentencia `if-else` y una sentencia `while`:

```

int a, b, c;
float x, y;
a = 10;
b = 20;
c = a + b * 2;
x = 3.14;
y = -2.5 + x;
if (a) { a = a - 1; } else { b = b + 1; }
while (c) { c = c / 2; }

```

Extras

Documentar el código. (0.25pts) — Listo.

Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos. (0.25pts) — No lo hice.